Welcome to the CUBETORY MAP EDITOR Documentation!

In here, you will find explanations about everything in the Editor, how to handle it, technical explanations and even a little “diary” of the development process.

You will find everything on the table of contents below. However, please read the following setup guide first.

SETUP GUIDE

First, you need to create and import your Cubetory map file. For this, launch the game and click on the map editor Button. Then, copy the original Cubetory map to your own maps.

Now, open the “My Maps” tab, select the one you have just copied and click on “Edit file”. This will open a new window of the file folder containing your map files. Copy the FULL file path into the “File Destination” Textbox in the Editor. Note that it must be the absolute path, so most likely, it will have to start with “C:\”. Finally, replace all backslashes (\) in the file path with normal slashes (/), so GoDot will not read them as string escapes.

Cubetory runs on six Json files that all handle a different part of the finished Map. These files are editable through their respective Buttons right beneath the file destination textbox.

The Meta file handles things like the map name and description; all other files should be self-explanatory.

It is recommended to start with the Meta file, then continue with the pit, stamper and fuel, and then finally add the upgrade tree, since that will require all other recipe IDs. Notice that the Terrain file is not listed here, as it works independently from all other files and can therefore be worked on whenever you want.

TABLE OF CONTENTS

1. The JSON Files

Cubetory reads its map data from six Json files that all contain a different part of what makes up the finished map. In the Editor, these files are editable through the six menus that open by the press of their corresponding buttons. Whilst every UI works a bit differently, there are some ground rules that count across all files:

1. IDs may not contain spaces and must be unique across all files.
2. All deletions of cube patterns or recipes are permanent and cannot be reversed.
3. Although most sliders have limits, you can always go into the JSON files yourself to adjust all values to your liking.

    a. When doing this, it Is strongly recommended to run the JSON through a prettifier first.
4. DO NOT TOUCH THE .bkp FILES. THEY ARE ONLY THERE TO SAVE THE EDITOR'S PROGRESS AND EVEN THE SLIGHTEST CHANGE MAY FULLY CORRUPT YOUR MAP.

When working on your map, make sure to playtest regularly. This is important because of how many error sources there are: Your own map may be at fault, but this Editor and even Cubetory itself may have bugs. If you run into issues, do not hesitate to ask the Discord. You can join it via the button in the Editor.

1.1 Meta

In this file, you can set your map name, description, resize limits, and sandbox mode. Once you are done, click the big "SAVE" Button at the bottom to write your changes into the JSON.

1.2 Fuel

In here, you can add new fuel recipes. The name is visible to players, the ID is only used by the editor, most notably the upgrades.json file.

The Cube Editor in the bottom left corner lets you set every cube face to any given color by clicking on the face to cycle through all variations. You can also choose the stamp decoration of the cube with the Dropdown-menu next to the pattern, which sets every face's stamp to the selected one. You can set as many fuels as you want, even though there are only three icons.

The list to the right of the Cube editor shows the ID, Name and Fuel Value of every fuel pattern you added.

1.3 Pit

The recipes_pit.json file contains all cube patterns that should reward the Player points for delivering them into the pit. Note that blank "pit cubes", so for example construction, thunder, music and so on, do NOT need an entry here to count as a Pit recipe, as they are already hard-coded to count towards their respective scores. Other stamps, like Bolts, Wire and Circuits, do need to be noted here before they count.

1.4 Stamper

All stamper recipes' input, output and byproducts information are stored here. Given how complicated the file's structure is, especially compared to others, the UI may appear overwhelming at first. New Recipes can be added by pressing the Button under the List of all recipes. (Via this List, recipes can also be permanently and irreversibly deleted) Pressing the "Add Recipe" Button will open a menu where you can configure the recipe's ID, Name, Ticks, Icon and Components. The ID is how the recipe can be accessed by other files, for example unlocked by the upgrade tree. Ticks means the time it takes for the stamper to process the recipe between receiving the input and delivering the output. One Tick is equal to one second.

To add a cube pattern, press the corresponding button at the bottom of the UI. If you want said cube to be packed, hold down shift whilst pressing the button.

When configuring an Input or output pattern, there is a "amount" option underneath the cube editor. This sets how many cubes of that kind need to be fed to the stamper or vice versa how many of that cube pattern the stamper will produce.

When configuring a Byproduct, this option is replaced by a "weight" option. This determines how likely the stamper is to output that specific cube as a byproduct. For example, a pattern with a weight of 1 is double as likely to be produced as one with a weight of 0.5. Although two weights of 1 behave the same as two weights of 0.5, it is recommended to use the ladder, as this allows the recipe menu in-game to show the player the actual percentage of getting each pattern.

Below this "amount" or "weight" option, there is a textbox where you can name this pattern. This name isn't used anywhere but this editor and will only show up in the list of byproducts, in- or outputs.

Next to the Button to add a byproduct, you will also find an "Amount" option, this sets how many byproducts are produced.

Once you are done configuring the recipe, click "Save", and it will take you back to the list of all recipes.

1.5 Terrain

The terrain file lets you add new land tiles. As you select your desired settings, more options like geyser color or ore yield may pop up. Click "add Terrain" once you're happy with the terrain tile, and "save all" to write the changes into the JSON.

When configuring the ore yield, a "weight" option will show up below the cube editor. This refers to the precent chance for a drill to output this cube from the ore patch.

1.6 Upgrades

This menu represents the Upgrade tree in-game. You can add Rows and Columns using the Buttons at the top and bottom of the tree. Every row has its cost, measured in cubes per minute. You can set this cost in the textbox at the very top of each row. Although you can theoretically add an unlimited number of rows and columns, it is recommended to keep your upgrade tree smaller or about as big as the one on the base-game map.

You can edit all upgrade slots by clicking on their respective button in the UI, and then setting the name, description and unlocks.

For the Unlocks, you must follow this syntax:

➔ Upgrade1, upgrade2

So, every Unlock must be separated by a comma and a space.

All elements that you want the player to have from the get-go can be listed in the “Unlocked from Start” Textbox. The same syntax must also be followed here.

All in-game Elements have a given name; you will find those alongside all other info on the full list.

To let the player unlock a pattern recipe that you set yourself, whether that be in the pit, the stamper, or the fuel, you can reference it through the ID that you gave said pattern.

2. Technical insights

Every JSON file only contains one Dictionary, whose size varies depending on how complicated its job is. As such, files like the upgrades or stamper are quite scuffed and chaotic.

This is also why a JSON prettifier is very helpful, because that mess of a dictionary is originally stored as a one-liner. Of course, you do not have to worry about any of this if you just use the Editor, but anyone who does want to scavenge that jungle of double-nested arrays will be glad to have at least some organization at hand. A lot of the structure in the JSON files served as a direct stencil for the UI in the Editor. This is not only due to the creator of the Editor being lazy (wink wink), but also because this creates a balance between you, the map creator, understanding the original nature of the JSON files and not having to deal with it, nonetheless.

The Meta file will not be explained any further, as it is literally just {“name”: “Map name”, “min_resize”: 0.1, “max_resize”: 2, “description”: “description”, “sandbox”: false}.

2.1 Cube Patterns

All Cube patterns are saved as a dictionary, containing two keys: “pattern” and “stamp”. The Pattern key refers to an Array of all six cube face’s colors as strings, whilst the stamp key stores one string describing the stamp of the cube.

2.2 Fuel

The fuel file stores every entry’s ID as a key, their corresponding values are dictionaries containing all other data: Name, Icon, Fuel value, and cube.

2.3 Pit

The Pit works almost the same as the fuel file, but it does not store a fuel value. Instead, it stores a “quantity” value. However, this value has no effect, which is also why you cannot edit it through the Editor. The only reason it is there is because the pit essentially works like a stamper in the code.

2.4 Stamper

The Stamper file stores all recipe’s IDs as Keys to a Dictionary storing the Recipe name, inputs, outputs, byproducts, icon, and ticks. The ladder is quite confusing, as it is unclear when the timer inside the stamper actually starts or stops. One tick means one belt push, and at normal game speed, this lasts one second.

2.5 Terrain

This file stores every chunk ID as a key, and every corresponding chunk data set as the respective value. There are actually two different ways to set a chunk feature: Either you set the “t” key to the corresponding string identifier, so for example “t”: “ore”, or you set t to “feature”, and add a key named “feature” and give it the corresponding integer value of a big Enum. For example, the garbage pit is only placeable with “t”: “feature” and “feature”: 31. The Editor uses a mix of these two, with every geyser and ore stored as string identifiers in a Dictionary.

2.6 Upgrades

The Upgrades File stores only three keys: “rates”, “start_unlocked”, and “rows”. The rates key is an array that holds an integer value for every column in the Upgrade tree, with each integer representing the rate of cubes per minute needed to unlock said upgrade. “start_unlocked” holds all elements that the player should have from the start in an array, and “rows” is a large array filled with even more arrays, each representing one row in the Upgrade tree. The Editor UI illustrates this nice and tidy as a visual representation of the Upgrade tree. In the code, it works with one big VBoxContainer that holds one HBoxContainer per row, and one Button inside each HBoxContainer per

column. The H- and VBoxContainers are nodes given by GoDot that align all their child nodes in the given Direction, H meaning horizontally and V meaning vertically.

4. Development Process

4.1 The Idea

When I discovered the Map Editor shortly after beating the base-game map and a user-made challenge, I was shocked to find out that using that “editor” literally just meant scavenging the JSON files and praying you do not break anything. So, I asked the developer on Discord If he had any plans to improve that “editor” to a real, user-friendly Editor. He replied, stating that he might work on better JSON validation, but a full editor is out of scope. Now, before I continue, I would like to clarify that I do not have anything against Cubetory or its developer, but I felt and still feel like Cubetory is rather expensive. Again, I should mention that my picture of a “fair price” is Terraria, and any Steam game selling for more than fifteen bucks automatically counts as too expensive for me. I am also not too avid of an automation gamer, but I still really like Cubetory. As such, I was also really happy that the Map editor would further justify Cubetory’s price tag. However, if that Editor would stay as “unfinished” as it is, players would not have a big reason to use it, and therefore, there would not be a lot of content on the Steam workshop. So, I wanted to help the Game and everyone who likes it by creating this Editor.

4.2 Development

Since I don’t have access to the program files of Cubetory, I had to learn every File’s Layout by analyzing its structure and editing specific details and then testing them in-game. This was, for the most part, far less tedious than it sounds. For each file, I created the UI following this process:

1. Analyzing the File structure.
2. Designing a fitting and easy-to-use UI that also works well with the file structure.
3. Coding the Backend of the UI and testing its final output through a print() method.
4. Coding the save_data function that writes the data into the JSON.
5. Connecting the UI to its Button in the main menu.

I first worked on the Meta file, due to how easy it is. Then, I moved on to the Upgrades file, after which all other files required cubes in some way, shape or form. So, I made a UI that lets you configure cube layouts and implemented said UI into all other file UIs. The only point where development slowed down was when I came to the terrain file. The terrain file runs on a large Enum containing all different geyser types. This Enum was still incomplete, wherefore I had to wait until the developer added all missing types. In the

meantime, I wrote this documentation. Finally, I tried adding a “packed” option to the cube editor but did not succeed. Upset that I couldn’t make a universally working UI, I settled on only making it work for the stamper. I am not particularly proud of this, but seeing as the packed cube can barely be used in any other form, I figured it won’t be too bad.

4.3 Roadmap

I don’t have big plans for this project, to be honest. However, I can guarantee support for bug fixes at the very least the next few years, and if Cubetory gets a big update, I will do my best to update this Editor as soon as possible to work with those new features. Until then, I wish all players and map creators a great time with Cubetory!

5. List of in-game elements

ID	Icon	Notes
belt		
nudger		
flipper		
launcher		
launcherdistance1		Adds one to the max launcher distance
launcherdistance2		
launcherdistance3		
splitter		
splitterwidth1		Adds one to the max splitter width
splitterwidth2		
loader		
loaderdistance1		Adds one to the max loader distance
loaderdistance2		
loaderdistance3		
tloader		
filter		
filterdistance1		Adds one to the max filter distance

filterstamps		Allows filtering for stamps
filtercolorcount		Allows filtering for color count
filterpolarized		Allows filtering for polarization
filtertags		Allows filtering for tags
filterdeepinspect		Allows deep inspect filtering
prioritizer		
mine		
biggermine		
biggestmine		
powerstation		
powerpole		
substation		
packer		
unpacker		
repacker		
polarizer		
pipe		
pipeinput		
pipeoutput		

pipepriority	none	Allows setting pipe in- and output priority
tagrouting		Allows pipes to route based on tags
sequencer		
sensor		
tagger		
untagger		
victory	none	Once the player unlocks this, they beat the map!

6. Changelog

29.5.26 Finished writing first version of Documentation.